LAB PROGRAM

Data Science with Python
NumPy • Pandas • Matplotlib • SciPy • Scikit-learn

40 programs - each with source code and executed output screenshots

Index

Prog.	Title
1	Subject-wise average and best subject (NumPy)
2	Average price of products sold (NumPy)
3	Average sale price of houses with more than 4 bedrooms
4	Total yearly sales and % increase from Q1 to Q4
5	Average fuel efficiency and % improvement between two models
6	Total cost of a purchase with discount and tax
7	Order analysis with Pandas
8	Top 5 best-selling products (Pandas)
9	Property analysis with Pandas
10	Line plot and bar plot of monthly sales (Matplotlib)
11	Line, scatter and bar plots of sales (Matplotlib)
12	Temperature line plot and rainfall scatter plot
13	Word frequency distribution of a text file
14	Frequency distribution of customer ages
15	Frequency distribution of likes on posts
16	Word frequency in customer reviews
17	Customer feedback text analysis with bar graph
18	Mean, median, std, boxplot, scatter and Q-Q plot (age vs %fat)
19	95% confidence interval for drug and placebo groups
20	A/B test: is there a significant difference in conversion rates?
21	Point estimate and confidence interval of rare-element concentration
22	Confidence interval of mean customer rating (Pandas)
23	Hypothesis test, p-value and visualization (treatment vs placebo)
24	KNN classifier for medical condition prediction
25	Decision Tree for Iris flower classification
26	Linear Regression for house price prediction
27	Logistic Regression for customer churn prediction
28	K-Means clustering for customer segmentation
29	Evaluation metrics: accuracy, precision, recall, F1-score
30	CART (decision tree regressor) for used-car price with decision path
31	Customer segmentation using K-Means clustering
32	Bivariate analysis and simple linear regression (house price vs size)
33	Multiple linear regression for car price with influential factors
34	KNN classifier for treatment outcome (Good / Bad)
35	K-Means clustering of retail customers by spending pattern
36	Variability of stock prices from a CSV file
37	Correlation between study time and exam scores
38	Temperature variability analysis for different cities
39	K-Means customer clustering with scatter-plot visualization
40	Soccer players analysis (CSV, Pandas, bar chart)

Program 1: Subject-wise average and best subject (NumPy)

Question: Marks of students (rows) in Math, Science, English, History (columns) are stored in student_scores. Find the average score of each subject and the subject with the highest average. (4x4 matrix assumed.)

Program (source code)

```
import numpy as np

# 4 students x 4 subjects (same code works for 32 x 4)
student_scores = np.array([[78, 85, 72, 66],
                           [92, 88, 95, 81],
                           [64, 70, 68, 59],
                           [85, 91, 79, 88]])
subjects = ["Math", "Science", "English", "History"]

avg_scores = student_scores.mean(axis=0)          # axis=0 -> column-wise mean
for subject, avg in zip(subjects, avg_scores):
    print(f"Average {subject:<8}: {avg:.2f}")

best = np.argmax(avg_scores)
print(f"\nSubject with highest average: {subjects[best]} ({avg_scores[best]:.2f})")
```

Output (executed)

Program 1 - Console Output

```
Average Math    : 79.75
Average Science : 83.50
Average English : 78.50
Average History : 73.50

Subject with highest average: Science (83.50)
```

Program 2: Average price of products sold (NumPy)

Question: Sales data of the last month is stored in a 3x3 NumPy array (each row = one product). Find the average price of all products sold.

Program (source code)

```
import numpy as np

# each row = a product, each column = price on a sales day
prices = np.array([[120.5, 130.0, 125.5],
                   [ 45.0,  50.0,  48.5],
                   [300.0, 310.0, 295.0]])

print("Price matrix:\n", prices)
print("\nAverage price of each product:", prices.mean(axis=1))
print(f"Average price of all products sold: {prices.mean():.2f}")
```

Output (executed)

Program 2 - Console Output

```
Price matrix:
 [[120.5 130.  125.5]
 [ 45.   50.   48.5]
 [300.  310.  295. ]]

Average price of each product: [125.33333333  47.83333333 301.66666667]
Average price of all products sold: 158.28
```

Program 3: Average sale price of houses with more than 4 bedrooms

Question: house_data holds bedrooms, square footage and sale price per house. Find the average sale price of houses having more than four bedrooms using NumPy.

Program (source code)

```
import numpy as np

# columns: bedrooms, square footage, sale price
house_data = np.array([[3, 1500, 250000],
                       [5, 2600, 480000],
                       [4, 2000, 320000],
                       [6, 3200, 610000],
                       [5, 2800, 520000],
                       [2,  900, 150000],
                       [7, 3500, 700000]])

mask = house_data[:, 0] > 4                      # boolean mask
big_houses = house_data[mask]
print("Houses with more than 4 bedrooms:\n", big_houses)
print(f"\nAverage sale price: {big_houses[:, 2].mean():,.2f}")
```

Output (executed)

```
Program 3 - Console Output

Houses with more than 4 bedrooms:
 [[     5   2600 480000]
 [     6   3200 610000]
 [     5   2800 520000]
 [     7   3500 700000]]

Average sale price: 577,500.00
```

Program 4: Total yearly sales and % increase from Q1 to Q4

Question: Quarterly sales are in sales_data. Calculate the total sales for the year and the percentage increase from the first quarter to the fourth quarter.

Program (source code)

```
import numpy as np

sales_data = np.array([25000, 28000, 31000, 36000])   # Q1..Q4

total_sales = sales_data.sum()
pct_increase = (sales_data[3] - sales_data[0]) / sales_data[0] * 100

print("Quarterly sales :", sales_data)
print("Total yearly sales :", total_sales)
print(f"% increase Q1 -> Q4 : {pct_increase:.2f}%")
```

Output (executed)

```
Program 4 - Console Output

Quarterly sales : [25000 28000 31000 36000]
Total yearly sales : 120000
% increase Q1 -> Q4 : 44.00%
```

Program 5: Average fuel efficiency and % improvement between two models

Question: fuel_efficiency stores the mpg of several car models. Calculate the average fuel efficiency and the percentage improvement between two car models.

Program (source code)

```
import numpy as np

fuel_efficiency = np.array([24.5, 28.0, 31.2, 22.8, 35.6])   # mpg of 5 models

print("Average fuel efficiency: %.2f mpg" % fuel_efficiency.mean())

old_model, new_model = fuel_efficiency[0], fuel_efficiency[4]   # compare model 1 and model 5
improvement = (new_model - old_model) / old_model * 100
print(f"Model 1 = {old_model} mpg, Model 5 = {new_model} mpg")
print(f"Percentage improvement: {improvement:.2f}%")
```

Output (executed)

Program 5 - Console Output

```
Average fuel efficiency: 28.42 mpg
Model 1 = 24.5 mpg, Model 5 = 35.6 mpg
Percentage improvement: 45.31%
```

Program 6: Total cost of a purchase with discount and tax

Question: Item prices and quantities are given in lists; discount and tax are percentages. Calculate the total cost payable by the customer.

Program (source code)

```
prices        = [45.50, 120.00, 30.25, 15.75]
quantities    = [2, 1, 4, 3]
discount_rate = 10      # percent
tax_rate      = 5       # percent

subtotal       = sum(p * q for p, q in zip(prices, quantities))
discount       = subtotal * discount_rate / 100
after_discount = subtotal - discount
tax            = after_discount * tax_rate / 100
total          = after_discount + tax

print(f"Subtotal          : {subtotal:8.2f}")
print(f"Discount ({discount_rate}%)     : {discount:8.2f}")
print(f"After discount    : {after_discount:8.2f}")
print(f"Tax ({tax_rate}%)          : {tax:8.2f}")
print(f"Total cost        : {total:8.2f}")
```

Output (executed)

Program 6 - Console Output

```
Subtotal          :   379.25
Discount (10%)     :    37.92
After discount    :   341.32
Tax (5%)          :    17.07
Total cost        :   358.39
```

Program 7: Order analysis with Pandas

Question: From order_data find: (1) total orders per customer, (2) average order quantity per product, (3) earliest and latest order dates.

Program (source code)

```
import pandas as pd

order_data = pd.DataFrame({
    "customer_id": [101, 102, 101, 103, 102, 101, 104, 103, 105, 102],
    "order_date": pd.to_datetime(["2026-01-05", "2026-01-07", "2026-01-12", "2026-01-15",
                                  "2026-01-20", "2026-02-01", "2026-02-03", "2026-02-10",
                                  "2026-02-14", "2026-02-25"]),
    "product_name": ["Laptop", "Mouse", "Keyboard", "Laptop", "Monitor",
                     "Mouse", "Keyboard", "Mouse", "Monitor", "Laptop"],
    "order_quantity": [1, 3, 2, 1, 2, 4, 1, 2, 1, 2]})

print("1. Total orders per customer:")
print(order_data.groupby("customer_id").size().rename("orders"))

print("\n2. Average order quantity per product:")
print(order_data.groupby("product_name")["order_quantity"].mean().round(2))

print("\n3. Earliest order date:", order_data["order_date"].min().date())
print("   Latest order date  :", order_data["order_date"].max().date())
```

Output (executed)

Program 7 - Console Output

```
1. Total orders per customer:
customer_id
101    3
102    3
103    2
104    1
105    1
Name: orders, dtype: int64

2. Average order quantity per product:
product_name
Keyboard    1.50
Laptop      1.33
Monitor     1.50
Mouse       3.00
Name: order_quantity, dtype: float64

3. Earliest order date: 2026-01-05
   Latest order date  : 2026-02-25
```

Program 8: Top 5 best-selling products (Pandas)

Question: Sales data of the past month is in a DataFrame. Find the top 5 products sold the most.

Program (source code)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(1)
products = ["Phone", "Laptop", "Headphones", "Charger", "Mouse", "Keyboard", "Monitor", "Webcam"]
sales = pd.DataFrame({"product": rng.choice(products, 60),
                      "quantity": rng.integers(1, 6, 60)})

top5 = sales.groupby("product")["quantity"].sum().sort_values(ascending=False).head(5)
print("Top 5 products sold the most (units):")
print(top5)
```

Output (executed)

Program 8 - Console Output

```
Top 5 products sold the most (units):
product
Headphones    37
Charger       34
Monitor       29
Webcam        23
Phone         22
Name: quantity, dtype: int64
```

Program 9: Property analysis with Pandas

Question: From property_data find: (1) average listing price per location, (2) number of properties with more than four bedrooms, (3) the property with the largest area.

Program (source code)

```
import pandas as pd

property_data = pd.DataFrame({
    "property_id": [1, 2, 3, 4, 5, 6, 7, 8],
    "location": ["Adyar", "Velachery", "Adyar", "T Nagar", "Velachery", "T Nagar", "Adyar", "Velachery"],
    "bedrooms": [3, 5, 4, 6, 2, 5, 3, 4],
    "area_sqft": [1400, 2600, 1900, 3300, 900, 2800, 1500, 2100],
    "listing_price": [8500000, 14500000, 11000000, 21000000, 5200000, 17500000, 9000000, 12500000]})

print("1. Average listing price per location:")
print(property_data.groupby("location")["listing_price"].mean().round(2))

print("\n2. Properties with more than 4 bedrooms:", (property_data["bedrooms"] > 4).sum())

largest = property_data.loc[property_data["area_sqft"].idxmax()]
print("\n3. Property with the largest area:")
print(largest)
```

Output (executed)

```
Program 9 - Console Output

1. Average listing price per location:
location
Adyar          9500000.00
T Nagar       19250000.00
Velachery     10733333.33
Name: listing_price, dtype: float64

2. Properties with more than 4 bedrooms: 3

3. Property with the largest area:
property_id             4
location          T Nagar
bedrooms                6
area_sqft            3300
listing_price    21000000
Name: 3, dtype: object
```

Program 10: Line plot and bar plot of monthly sales (Matplotlib)

Question: Develop a Python program to create (1) a line plot and (2) a bar plot of the monthly sales data.

Program (source code)

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
sales  = [210, 240, 275, 260, 300, 340, 330, 360, 390, 410, 450, 520]
print("Monthly sales data loaded for", len(months), "months")

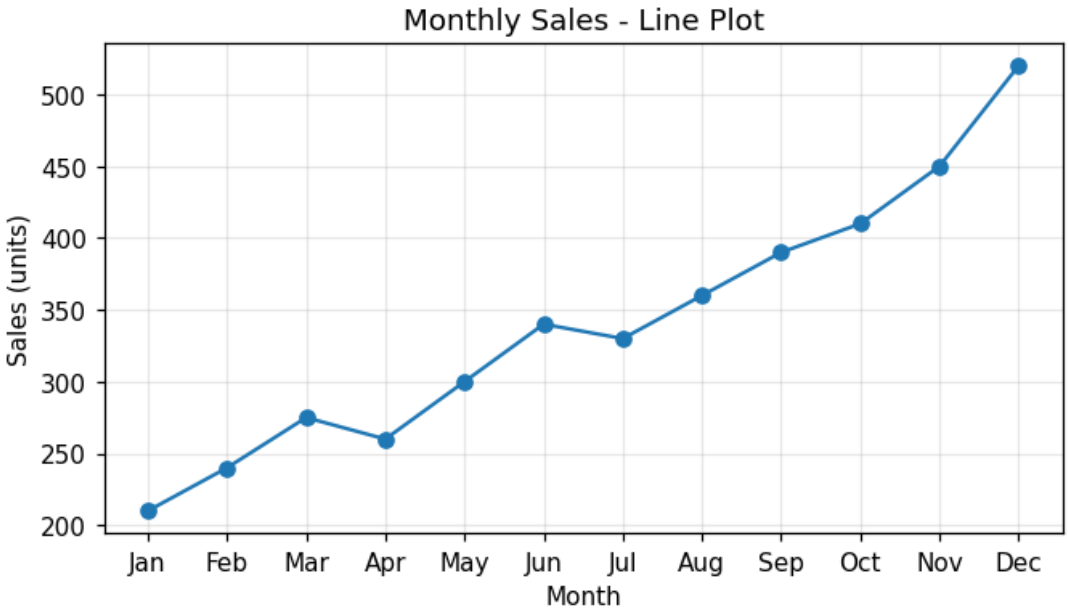
# 1. Line plot
plt.figure(figsize=(7, 3.6))
plt.plot(months, sales, marker="o", color="tab:blue")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month"); plt.ylabel("Sales (units)")
plt.grid(True, alpha=0.3)
plt.show()

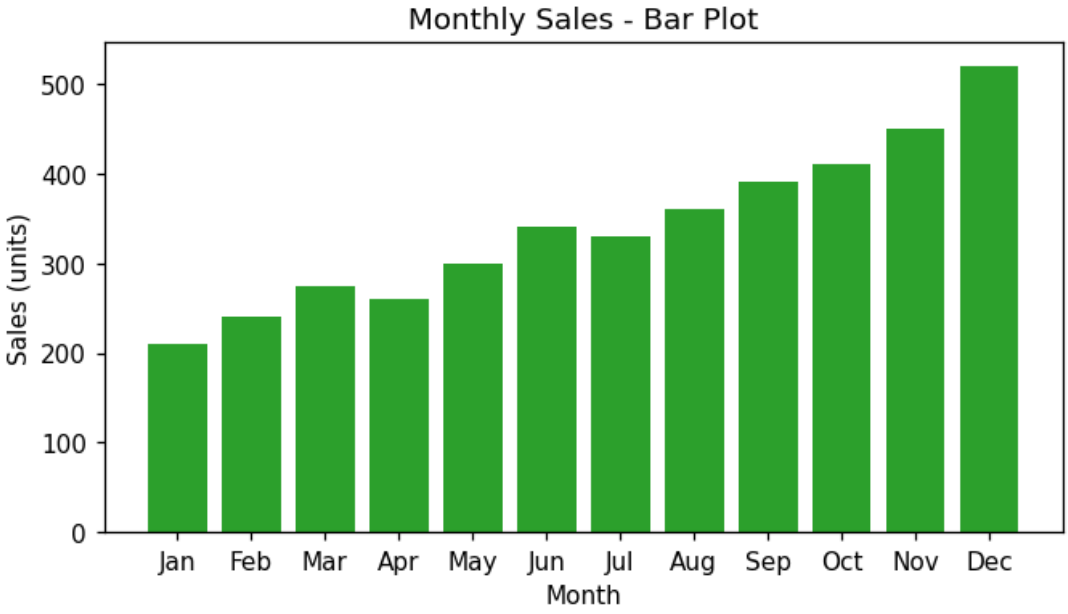
# 2. Bar plot
plt.figure(figsize=(7, 3.6))
plt.bar(months, sales, color="tab:green")
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month"); plt.ylabel("Sales (units)")
plt.show()
```

Output (executed)

```
Program 10 - Console Output

Monthly sales data loaded for 12 months
```


Monthly Sales - Line Plot
Sales (units)
500
450
400
350
300
250
200
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month


Monthly Sales - Bar Plot
Sales (units)
500
400
300
200
100
0
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month

Program 11: Line, scatter and bar plots of sales (Matplotlib)

Question: (1) Line plot of sales in a month, (2) scatter plot with a trend line to predict sales, (3) bar plot of the monthly sales data.

Program (source code)

```
import numpy as np
import matplotlib.pyplot as plt

days = np.arange(1, 31)
rng = np.random.default_rng(3)
daily_sales = 50 + 2.5 * days + rng.normal(0, 8, 30)       # upward trend + noise

# 1. Line plot
plt.figure(figsize=(7, 3.4))
plt.plot(days, daily_sales, color="tab:blue")
plt.title("Daily Sales in a Month (Line Plot)"); plt.xlabel("Day"); plt.ylabel("Sales")
plt.show()

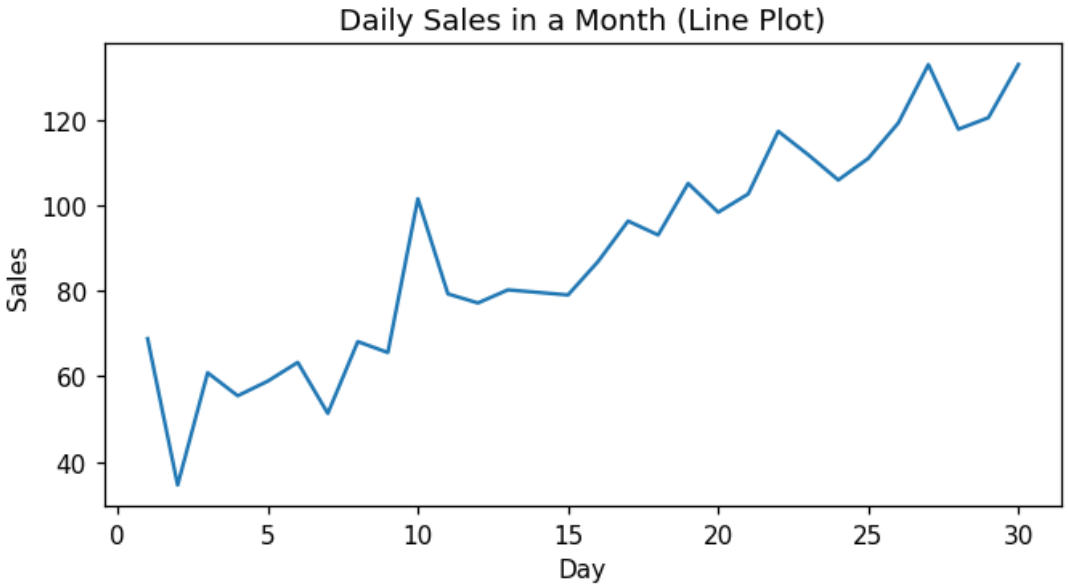
# 2. Scatter plot with linear trend line -> predict sales on day 31
slope, intercept = np.polyfit(days, daily_sales, 1)
plt.figure(figsize=(7, 3.4))
plt.scatter(days, daily_sales, color="tab:orange", label="Actual sales")
plt.plot(days, slope * days + intercept, color="red", label="Trend line")
plt.title("Daily Sales (Scatter Plot + Trend)"); plt.xlabel("Day"); plt.ylabel("Sales"); plt.legend()
plt.show()
print(f"Predicted sales for day 31: {slope * 31 + intercept:.1f}")

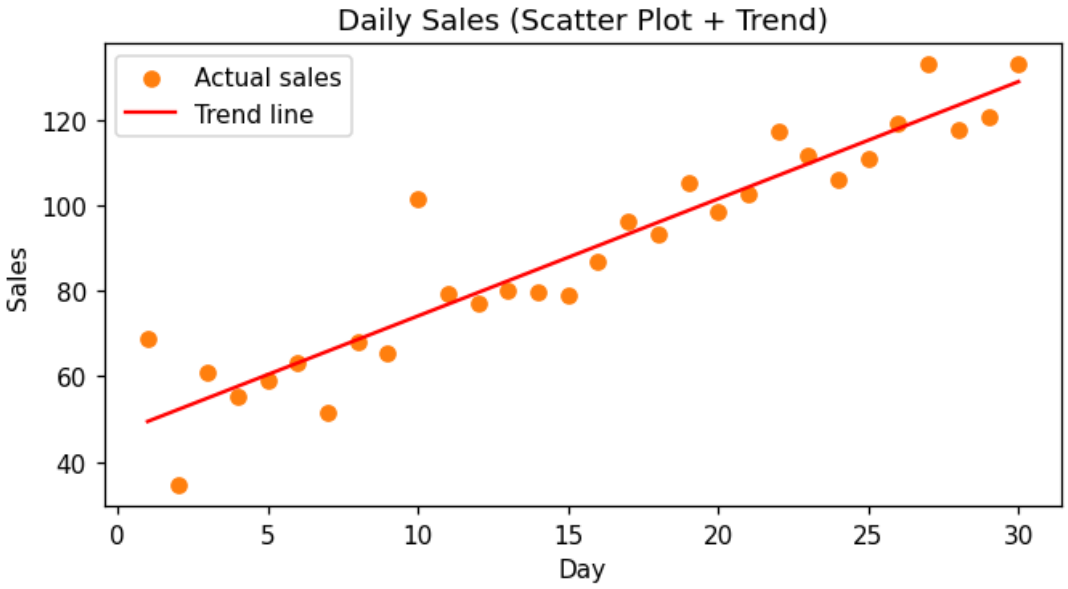
# 3. Bar plot of monthly sales
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
monthly = [1800, 2100, 2600, 2400, 3000, 3300]
plt.figure(figsize=(7, 3.4))
plt.bar(months, monthly, color="tab:purple")
plt.title("Monthly Sales (Bar Plot)"); plt.xlabel("Month"); plt.ylabel("Sales")
plt.show()
```

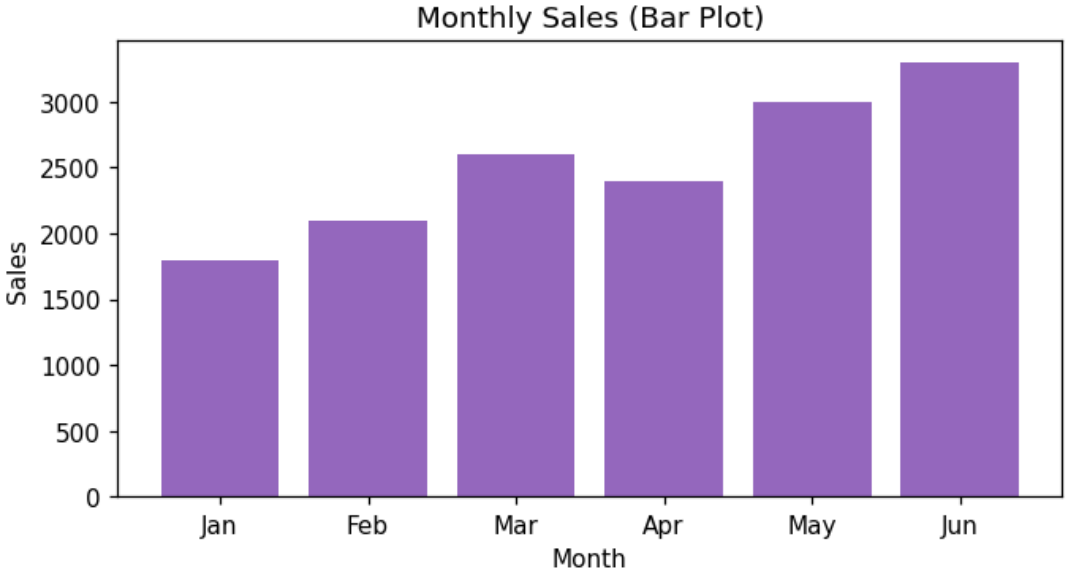
Output (executed)

```
Program 11 - Console Output

Predicted sales for day 31: 131.7
```







Program 12: Temperature line plot and rainfall scatter plot

Question: Develop a program to create (1) a line plot of monthly temperature and (2) a scatter plot of monthly rainfall for a city.

Program (source code)

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
temperature = [24, 26, 29, 32, 35, 34, 32, 31, 30, 28, 26, 24]          # deg C
rainfall    = [24, 7, 15, 25, 52, 53, 83, 124, 118, 267, 309, 139]       # mm
print("Average temperature: %.1f C" % (sum(temperature) / 12))
print("Total rainfall     : %d mm" % sum(rainfall))

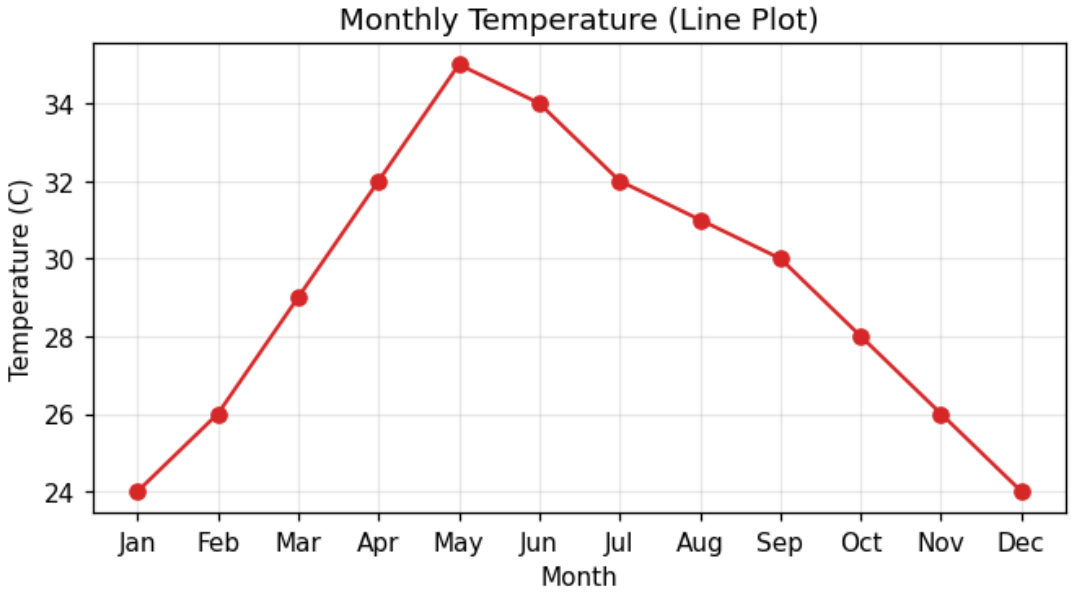
# 1. Line plot of temperature
plt.figure(figsize=(7, 3.4))
plt.plot(months, temperature, marker="o", color="tab:red")
plt.title("Monthly Temperature (Line Plot)"); plt.xlabel("Month"); plt.ylabel("Temperature (C)")
plt.grid(True, alpha=0.3)
plt.show()

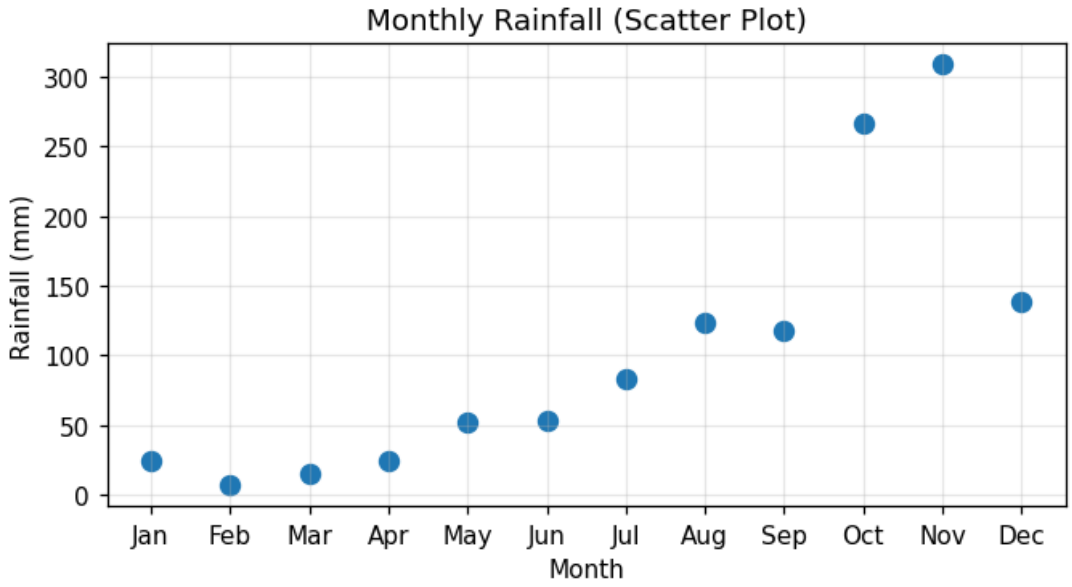
# 2. Scatter plot of rainfall
plt.figure(figsize=(7, 3.4))
plt.scatter(months, rainfall, color="tab:blue", s=60)
plt.title("Monthly Rainfall (Scatter Plot)"); plt.xlabel("Month"); plt.ylabel("Rainfall (mm)")
plt.grid(True, alpha=0.3)
plt.show()
```

Output (executed)

```
Program 12 - Console Output

Average temperature: 29.2 C
Total rainfall     : 1216 mm
```





Program 13: Word frequency distribution of a text file

Question: Read sample_text.txt, process the text and generate the frequency distribution of its words.

Program (source code)

```
import re
from collections import Counter

# create the sample file
text = ("Data science is the study of data. Data scientists use Python to clean data, "
        "build models and share insights. Python is popular because Python is simple, "
        "and the libraries for data analysis are powerful.")
with open("sample_text.txt", "w") as f:
    f.write(text)

# read, clean and tokenize
with open("sample_text.txt") as f:
    raw = f.read()
words = re.findall(r"[a-z]+", raw.lower())

freq = Counter(words)
print("Total words :", len(words))
print("Unique words:", len(freq))
print(f"\n{'Word':<12}{'Frequency'}")
for word, count in freq.most_common(8):
    print(f"{word:<12}{count}")
```

Output (executed)

Program 13 - Console Output

```
Total words : 34
Unique words: 24

Word        Frequency
data        5
is          3
python      3
the         2
and         2
science     1
study       1
of          1
```

Program 14: Frequency distribution of customer ages

Question: Find the frequency distribution of the ages of customers who purchased in the past month (Pandas).

Program (source code)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(5)
customers = pd.DataFrame({"customer_id": range(1, 101),
                          "age": rng.normal(35, 10, 100).round().clip(18, 65).astype(int)})

bins = [17, 25, 35, 45, 55, 65]
labels = ["18-25", "26-35", "36-45", "46-55", "56-65"]
customers["age_group"] = pd.cut(customers["age"], bins=bins, labels=labels)

freq = customers["age_group"].value_counts().sort_index()
dist = pd.DataFrame({"Frequency": freq, "Percent": (freq / freq.sum() * 100).round(1)})
print("Frequency distribution of customer ages:")
print(dist)
```

Output (executed)

Program 14 - Console Output

```
Frequency distribution of customer ages:
           Frequency  Percent
age_group
18-25             27     27.0
26-35             39     39.0
36-45             26     26.0
46-55              7      7.0
56-65              1      1.0
```

Program 15: Frequency distribution of likes on posts

Question: Calculate the frequency distribution of likes among posts of a social media platform.

Program (source code)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
posts = pd.DataFrame({"post_id": range(1, 81),
                      "likes": rng.gamma(2.0, 60, 80).astype(int)})

bins = [0, 50, 100, 200, 400, 1000]
labels = ["0-50", "51-100", "101-200", "201-400", "401+"]
posts["range"] = pd.cut(posts["likes"], bins=bins, labels=labels, include_lowest=True)

freq = posts["range"].value_counts().sort_index()
table = pd.DataFrame({"Posts": freq,
                      "Relative freq": (freq / len(posts)).round(3),
                      "Cumulative": freq.cumsum()})
print("Frequency distribution of likes:")
print(table)
```

Output (executed)

```
Program 15 - Console Output

Frequency distribution of likes:
          Posts  Relative freq  Cumulative
range
0-50         11          0.138          11
51-100       25          0.312          36
101-200      29          0.362          65
201-400      15          0.188          80
401+          0          0.000          80
```

Program 16: Word frequency in customer reviews

Question: Calculate the frequency distribution of words in a customer reviews dataset.

Program (source code)

```
import re
import pandas as pd
from collections import Counter

reviews = pd.DataFrame({"review": [
    "Great product, works perfectly and the battery is great",
    "Poor quality, the battery drains fast",
    "Excellent value for money, great quality",
    "The product arrived late but the quality is good",
    "Not worth the price, poor battery life",
    "Great quality and fast delivery",
    "Good product, good price, will buy again",
    "Battery life is excellent and the screen is great"]})

stop_words = {"the", "is", "and", "a", "but", "for", "not", "will", "of", "to"}
words = []
for review in reviews["review"]:
    tokens = re.findall(r"[a-z]+", review.lower())
    words += [w for w in tokens if w not in stop_words]

freq = Counter(words)
print(f"{'Word':<10}{'Count':>5}")
for word, count in freq.most_common(10):
    print(f"{word:<10}{count:>5}")
```

Output (executed)

```
Program 16 - Console Output

Word       Count
great          5
battery        4
quality        4
product        3
good           3
poor           2
fast           2
excellent      2
price          2
life           2
```

Program 17: Customer feedback text analysis with bar graph

Question: Load data.csv, preprocess (remove punctuation, lowercase, remove stop words), find word frequencies, display top N words (N from user) and plot a bar graph.

Program (source code)

```
import re
import random
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter

# ---- create data.csv with a "feedback" column ----
random.seed(0)
comments = ["The delivery was fast and the packaging is great!",
            "Customer support is slow, and the app keeps crashing.",
            "Great prices, great quality; I love this store!",
            "The app is easy to use, but delivery was late.",
            "Poor packaging and the product is damaged.",
            "Fast delivery and friendly support, will order again!",
            "The quality is good, however the price is high."]
pd.DataFrame({"feedback": [random.choice(comments) for _ in range(60)]}).to_csv("data.csv", index=False)

# ---- load and preprocess ----
df = pd.read_csv("data.csv")
stop_words = {"the", "and", "is", "was", "a", "to", "of", "but", "i", "this", "it", "however", "will"}

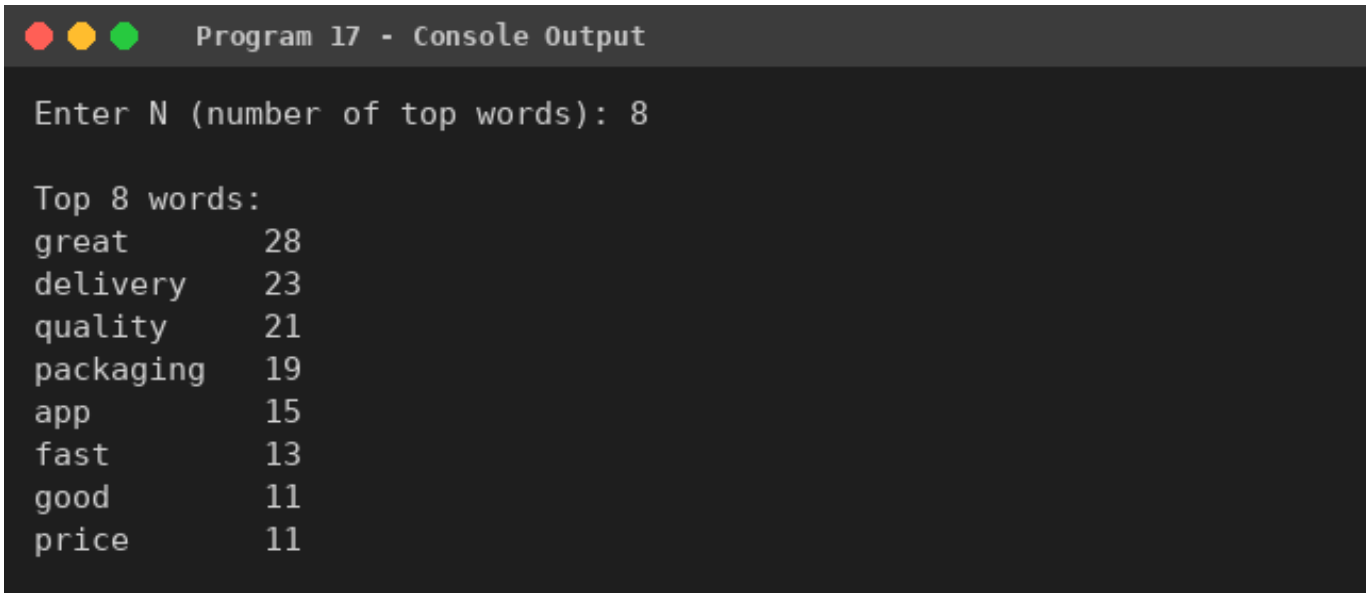
def preprocess(text):
    text = re.sub(r"[^\w\s]", "", text.lower())            # remove punctuation, lowercase
    return [w for w in text.split() if w not in stop_words]  # remove stop words

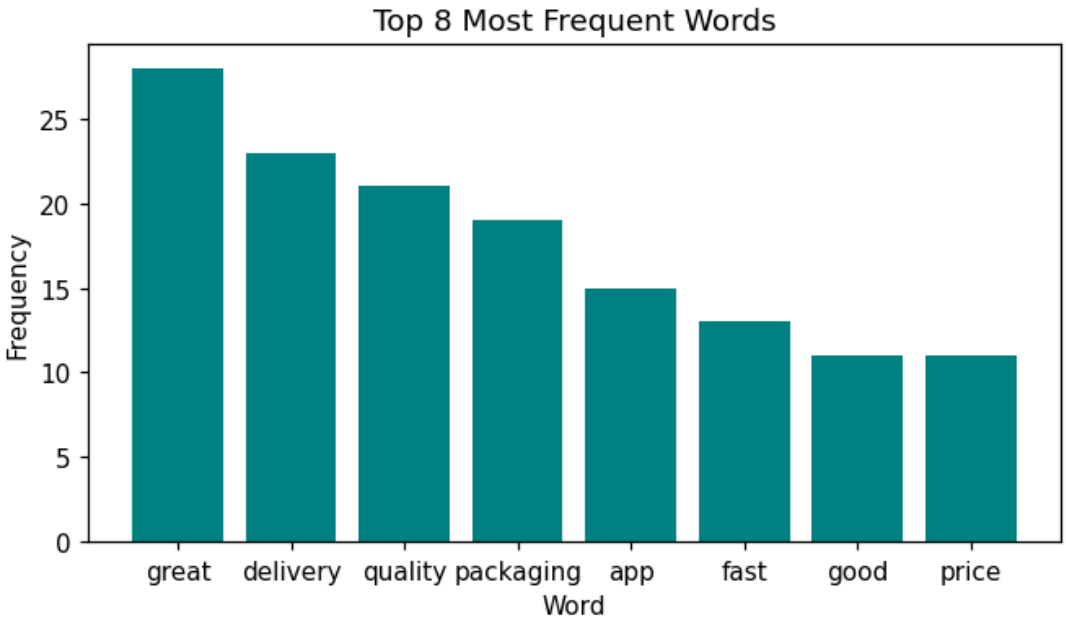
words = [w for comment in df["feedback"] for w in preprocess(comment)]
freq = Counter(words)

N = int(input("Enter N (number of top words): "))
top = freq.most_common(N)
print(f"\nTop {N} words:")
for word, count in top:
    print(f"{word:<12}{count}")

plt.figure(figsize=(7, 3.6))
plt.bar([w for w, _ in top], [c for _, c in top], color="teal")
plt.title(f"Top {N} Most Frequent Words"); plt.xlabel("Word"); plt.ylabel("Frequency")
plt.show()
```

Output (executed)





Program 18: Mean, median, std, boxplot, scatter and Q-Q plot (age vs %fat)

Question: For 18 adults' age and %fat: calculate mean, median and standard deviation using Pandas; draw boxplots; draw a scatter plot and a Q-Q plot of the two variables.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

age = [23, 23, 27, 27, 39, 41, 47, 49, 50, 52, 54, 54, 56, 57, 58, 58, 60, 61]
fat = [9.5, 26.5, 7.8, 17.8, 31.4, 25.9, 27.4, 27.2, 31.2, 34.6, 42.5, 28.8,
       33.4, 30.2, 34.1, 32.9, 41.2, 35.7]
df = pd.DataFrame({"age": age, "%fat": fat})

print(df.agg(["mean", "median", "std"]).round(2))

# Boxplots
plt.figure(figsize=(7, 3.6))
plt.subplot(1, 2, 1); plt.boxplot(df["age"]); plt.title("Boxplot of age"); plt.xticks([1], ["age"])
plt.subplot(1, 2, 2); plt.boxplot(df["%fat"]); plt.title("Boxplot of %fat"); plt.xticks([1], ["%fat"])
plt.tight_layout(); plt.show()

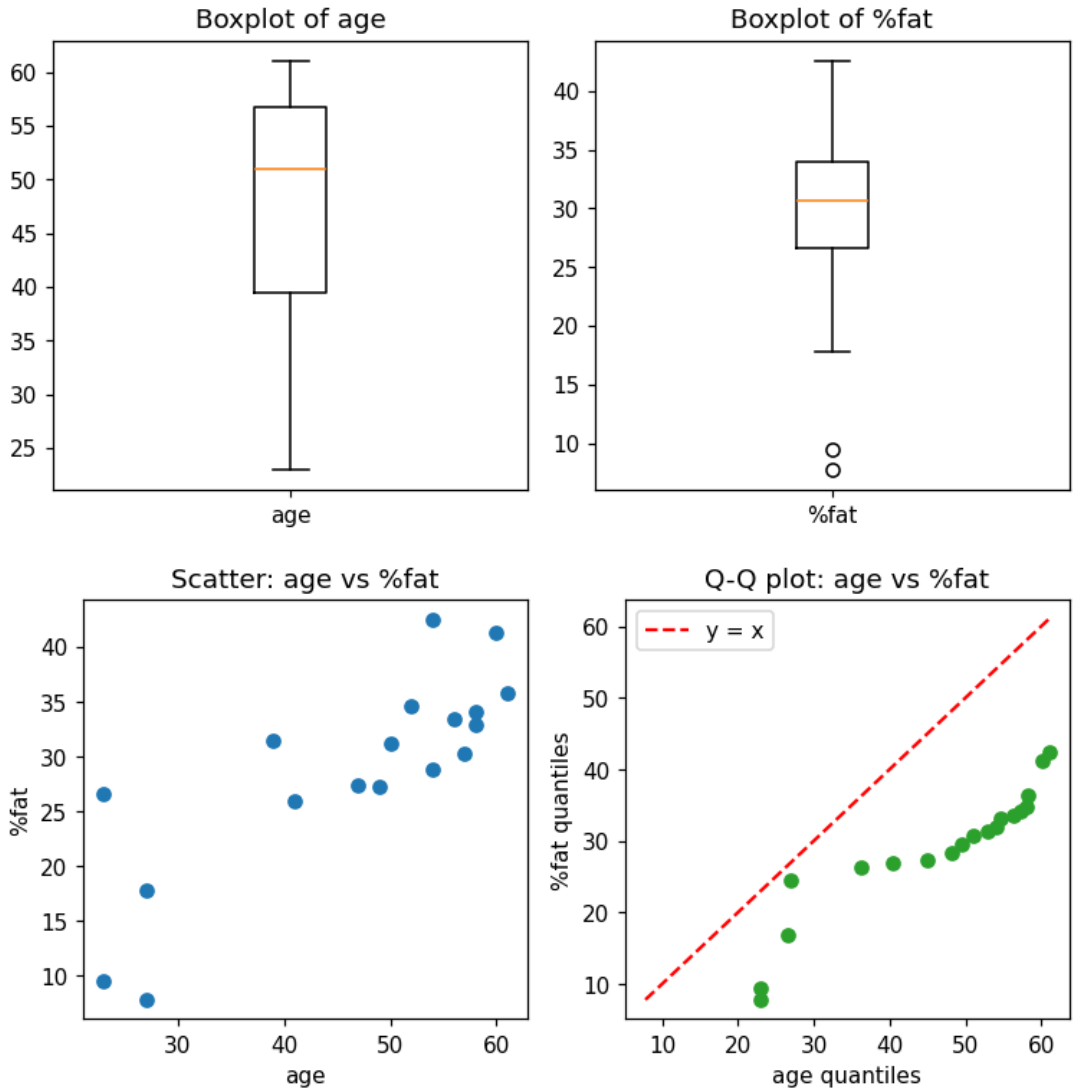
# Scatter plot and Q-Q plot
plt.figure(figsize=(7, 3.6))
plt.subplot(1, 2, 1)
plt.scatter(df["age"], df["%fat"], color="tab:blue")
plt.title("Scatter: age vs %fat"); plt.xlabel("age"); plt.ylabel("%fat")

plt.subplot(1, 2, 2)
p = np.linspace(0, 100, 19)                       # compare quantiles of the two variables
qa, qf = np.percentile(df["age"], p), np.percentile(df["%fat"], p)
plt.scatter(qa, qf, color="tab:green")
lims = [min(qa.min(), qf.min()), max(qa.max(), qf.max())]
plt.plot(lims, lims, "r--", label="y = x")
plt.title("Q-Q plot: age vs %fat"); plt.xlabel("age quantiles"); plt.ylabel("%fat quantiles")
plt.legend(); plt.tight_layout(); plt.show()
```

Output (executed)

```
Program 18 - Console Output

          age   %fat
mean    46.44  28.78
median  51.00  30.70
std     13.22   9.25
```



Program 19: 95% confidence interval for drug and placebo groups

Question: 50 patients (25 drug, 25 placebo). Find the 95% confidence interval of the mean reduction in blood pressure for each group.

Program (source code)

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
drug    = rng.normal(15, 6, 25)       # reduction in blood pressure (mmHg)
placebo = rng.normal(5, 5, 25)

def confidence_interval(data, confidence=0.95):
    n, mean, se = len(data), data.mean(), stats.sem(data)
    low, high = stats.t.interval(confidence, df=n - 1, loc=mean, scale=se)
    return mean, low, high

for name, group in [("Drug", drug), ("Placebo", placebo)]:
    mean, low, high = confidence_interval(group)
    print(f"{name:<8}: mean reduction = {mean:5.2f} mmHg, 95% CI = ({low:5.2f}, {high:5.2f})")
```

Output (executed)

```
Program 19 - Console Output

Drug    : mean reduction = 14.79 mmHg, 95% CI = (12.73, 16.85)
Placebo : mean reduction =  6.09 mmHg, 95% CI = ( 4.66,  7.52)
```

Program 20: A/B test: is there a significant difference in conversion rates?

Question: Conversion-rate data for website designs A and B is collected. Test whether the mean conversion rates differ significantly.

Program (source code)

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(8)
design_A = rng.normal(0.112, 0.020, 30)      # daily conversion rates
design_B = rng.normal(0.128, 0.020, 30)

print(f"Mean conversion A: {design_A.mean():.4f}")
print(f"Mean conversion B: {design_B.mean():.4f}")

t_stat, p_value = stats.ttest_ind(design_A, design_B, equal_var=False)   # Welch t-test
alpha = 0.05
print(f"\nt-statistic = {t_stat:.3f}, p-value = {p_value:.4f}")
if p_value < alpha:
    print("Reject H0: there IS a statistically significant difference between designs A and B.")
else:
    print("Fail to reject H0: no significant difference between designs A and B.")
```

Output (executed)

```
Program 20 - Console Output

Mean conversion A: 0.1100
Mean conversion B: 0.1261

t-statistic = -2.673, p-value = 0.0098
Reject H0: there IS a statistically significant difference between designs A and B.
```

Program 21: Point estimate and confidence interval of rare-element concentration

Question: Read rare_elements.csv; the user supplies sample size, confidence level and desired precision. Compute the point estimate and confidence interval with NumPy.

Program (source code)

```
import numpy as np
from scipy import stats

# ---- create rare_elements.csv (300 measurements) ----
rng = np.random.default_rng(7)
np.savetxt("rare_elements.csv", rng.normal(12.5, 2.2, 300), header="concentration",
           comments="", fmt="%.3f")

data = np.loadtxt("rare_elements.csv", skiprows=1)

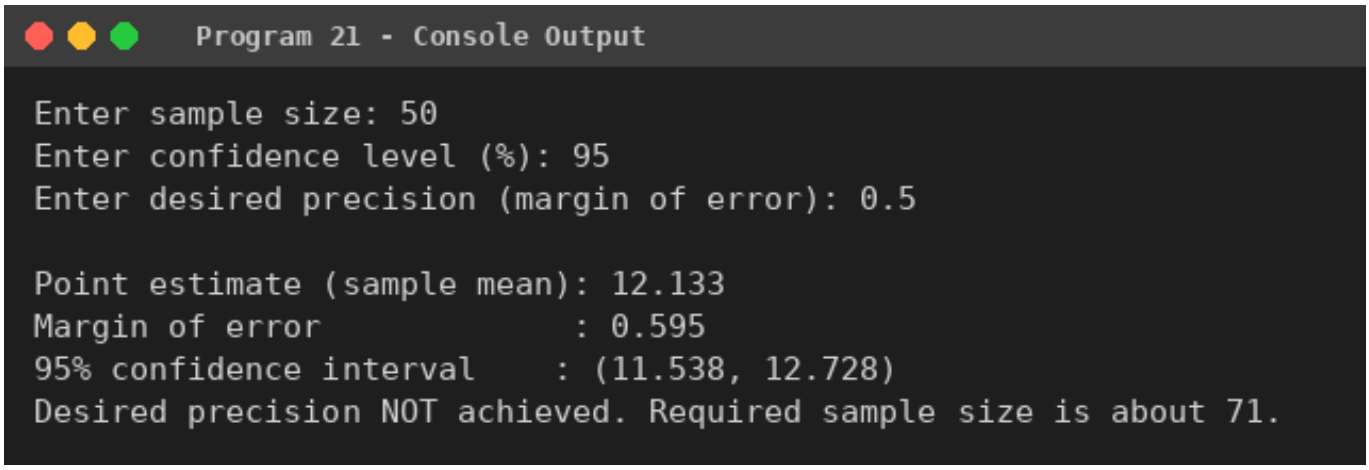
n         = int(input("Enter sample size: "))
conf      = float(input("Enter confidence level (%): ")) / 100
precision = float(input("Enter desired precision (margin of error): "))

sample = rng.choice(data, size=n, replace=False)
mean, s = sample.mean(), sample.std(ddof=1)
t_crit  = stats.t.ppf((1 + conf) / 2, df=n - 1)
margin  = t_crit * s / np.sqrt(n)

print(f"\nPoint estimate (sample mean): {mean:.3f}")
print(f"Margin of error             : {margin:.3f}")
print(f"{conf*100:.0f}% confidence interval    : ({mean - margin:.3f}, {mean + margin:.3f})")

if margin <= precision:
    print("Desired precision achieved.")
else:
    needed = int(np.ceil((t_crit * s / precision) ** 2))
    print(f"Desired precision NOT achieved. Required sample size is about {needed}.")
```

Output (executed)



Program 22: Confidence interval of mean customer rating (Pandas)

Question: Using customer_reviews.csv, compute a confidence interval to estimate the true population mean rating of a product category.

Program (source code)

```
import numpy as np
import pandas as pd
from scipy import stats

# ---- create customer_reviews.csv ----
rng = np.random.default_rng(21)
pd.DataFrame({"category": rng.choice(["Electronics", "Books"], 200),
              "rating": rng.choice([1, 2, 3, 4, 5], 200, p=[0.05, 0.10, 0.20, 0.35, 0.30])}
             ).to_csv("customer_reviews.csv", index=False)

df = pd.read_csv("customer_reviews.csv")
ratings = df.loc[df["category"] == "Electronics", "rating"]

n, mean, sem = len(ratings), ratings.mean(), ratings.sem()
low, high = stats.t.interval(0.95, df=n - 1, loc=mean, scale=sem)

print(f"Reviews analysed      : {n}")
print(f"Sample mean rating    : {mean:.3f}")
print(f"95% confidence interval: ({low:.3f}, {high:.3f})")
print(f"Customer satisfaction (rating >= 4): {(ratings >= 4).mean() * 100:.1f}%")
```

Output (executed)

```
Program 22 - Console Output

Reviews analysed      : 96
Sample mean rating    : 3.760
95% confidence interval: (3.537, 3.984)
Customer satisfaction (rating >= 4): 65.6%
```

Program 23: Hypothesis test, p-value and visualization (treatment vs placebo)

Question: Analyse clinical-trial data of a control (placebo) group and a treatment group using a hypothesis test, calculate the p-value and visualize the data and p-value with Matplotlib.

Program (source code)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(10)
placebo   = rng.normal(120, 10, 40)      # e.g. blood-pressure after trial
treatment = rng.normal(115, 10, 40)

t_stat, p_value = stats.ttest_ind(treatment, placebo)
print(f"Placebo mean   = {placebo.mean():.2f}")
print(f"Treatment mean = {treatment.mean():.2f}")
print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
print("Result:", "statistically significant (p < 0.05)" if p_value < 0.05 else "not significant")

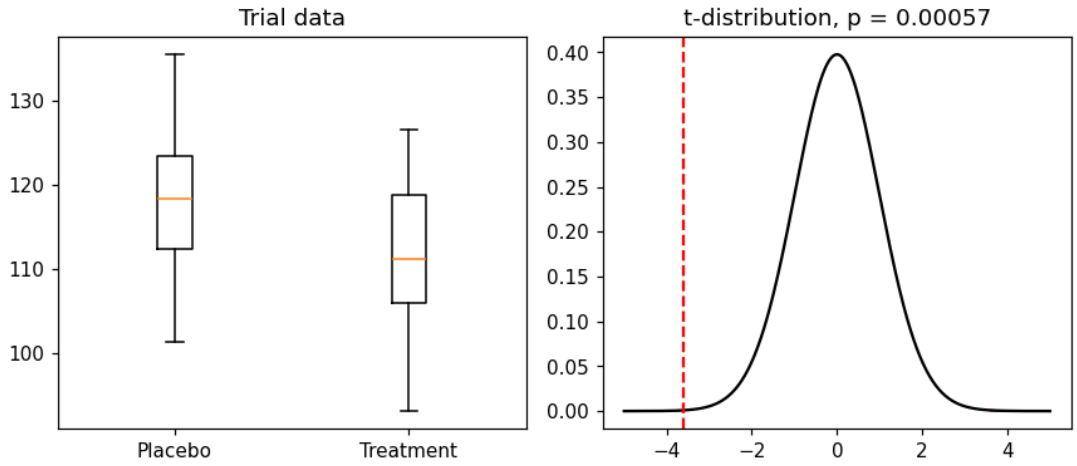
fig, ax = plt.subplots(1, 2, figsize=(8, 3.6))
ax[0].boxplot([placebo, treatment], tick_labels=["Placebo", "Treatment"])
ax[0].set_title("Trial data")

df = len(placebo) + len(treatment) - 2
x = np.linspace(-5, 5, 400)
ax[1].plot(x, stats.t.pdf(x, df), color="black")
ax[1].fill_between(x, stats.t.pdf(x, df), where=(abs(x) >= abs(t_stat)), color="red", alpha=0.5)
ax[1].axvline(t_stat, color="red", linestyle="--")
ax[1].set_title(f"t-distribution, p = {p_value:.5f}")
plt.tight_layout(); plt.show()
```

Output (executed)

```
Program 23 - Console Output

Placebo mean   = 118.06
Treatment mean = 111.51
t-statistic = -3.596, p-value = 0.00057
Result: statistically significant (p < 0.05)
```



Program 24: KNN classifier for medical condition prediction

Question: Predict whether a patient has a medical condition from symptom features and a user-supplied k, using the KNN classifier of scikit-learn.

Program (source code)

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

# ---- labelled patient data: temperature, heart rate, age ----
rng = np.random.default_rng(0)
X = np.column_stack([np.r_[rng.normal(36.8, 0.4, 50), rng.normal(38.6, 0.6, 50)],
                     np.r_[rng.normal(76, 8, 50), rng.normal(96, 10, 50)],
                     rng.integers(20, 80, 100)])
y = np.r_[np.zeros(50), np.ones(50)]            # 0 = no condition, 1 = condition

scaler = StandardScaler().fit(X)
features = [float(v) for v in input("Enter temperature, heart_rate, age (comma separated): ").split(",")]
k = int(input("Enter value of k: "))

knn = KNeighborsClassifier(n_neighbors=k).fit(scaler.transform(X), y)
new_patient = scaler.transform([features])
pred = knn.predict(new_patient)[0]
prob = knn.predict_proba(new_patient)[0]

print("\nPrediction:", "Patient HAS the condition (1)" if pred == 1 else "Patient has NO condition (0)")
print(f"Probability of condition: {prob[1]:.2f}")
```

Output (executed)

```
Program 24 - Console Output

Enter temperature, heart_rate, age (comma separated): 38.9, 98, 52
Enter value of k: 5

Prediction: Patient HAS the condition (1)
Probability of condition: 1.00
```

Program 25: Decision Tree for Iris flower classification

Question: Load the Iris dataset, accept sepal/petal measurements of a new flower and predict its species with a Decision Tree classifier.

Program (source code)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
                                                    test_size=0.2, random_state=42)
model = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
print(f"Model accuracy on test data: {model.score(X_test, y_test):.2f}\n")

sl = float(input("Enter sepal length (cm): "))
sw = float(input("Enter sepal width  (cm): "))
pl = float(input("Enter petal length (cm): "))
pw = float(input("Enter petal width  (cm): "))

pred = model.predict([[sl, sw, pl, pw]])[0]
print("\nPredicted species:", iris.target_names[pred])
```

Output (executed)

```
Program 25 - Console Output

Model accuracy on test data: 1.00

Enter sepal length (cm): 6.1
Enter sepal width  (cm): 2.8
Enter petal length (cm): 4.7
Enter petal width  (cm): 1.2

Predicted species: versicolor
```

Program 26: Linear Regression for house price prediction

Question: Accept the features (area, bedrooms, location score) of a new house and predict its price using scikit-learn's linear regression.

Program (source code)

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# ---- house dataset ----
rng = np.random.default_rng(1)
n = 150
houses = pd.DataFrame({"area": rng.integers(600, 3500, n),
                       "bedrooms": rng.integers(1, 6, n),
                       "location_score": rng.integers(1, 6, n)})
houses["price"] = (60 * houses["area"] + 15000 * houses["bedrooms"] +
                   25000 * houses["location_score"] + rng.normal(0, 20000, n))

X, y = houses[["area", "bedrooms", "location_score"]], houses["price"]
model = LinearRegression().fit(X, y)
print(f"Model R-squared: {r2_score(y, model.predict(X)):.3f}")

area = float(input("Enter area (sq ft): "))
bedrooms = int(input("Enter number of bedrooms: "))
location = int(input("Enter location score (1-5): "))

new_house = pd.DataFrame([[area, bedrooms, location]], columns=X.columns)
print(f"\nPredicted price of the house: ${model.predict(new_house)[0]:,.2f}")
```

Output (executed)

```
Program 26 - Console Output

Model R-squared: 0.923
Enter area (sq ft): 1800
Enter number of bedrooms: 3
Enter location score (1-5): 4

Predicted price of the house: $249,684.43
```

Program 27: Logistic Regression for customer churn prediction

Question: Accept the features of a new customer and predict whether the customer will churn using logistic regression.

Program (source code)

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

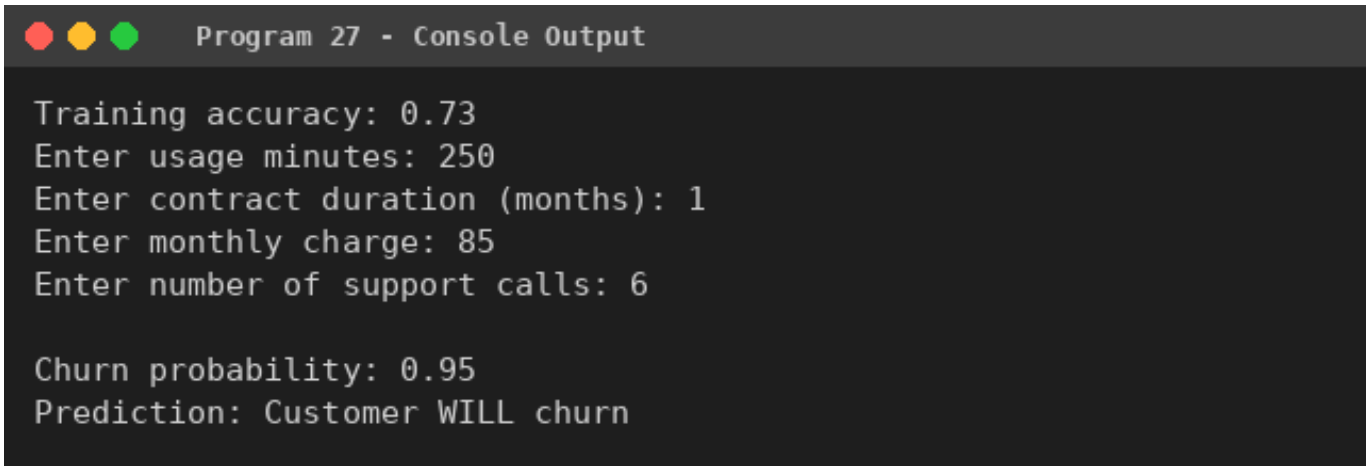
# ---- past customer data ----
rng = np.random.default_rng(2)
n = 400
df = pd.DataFrame({"usage_minutes": rng.normal(300, 100, n),
                   "contract_months": rng.choice([1, 12, 24], n),
                   "monthly_charge": rng.normal(60, 20, n),
                   "support_calls": rng.integers(0, 8, n)})
score = (0.02 * (df["monthly_charge"] - 60) + 0.5 * (df["support_calls"] - 3)
         - 0.08 * (df["contract_months"] - 12) - 0.004 * (df["usage_minutes"] - 300))
df["churn"] = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)

X, y = df.drop(columns="churn"), df["churn"]
model = make_pipeline(StandardScaler(), LogisticRegression()).fit(X, y)
print(f"Training accuracy: {model.score(X, y):.2f}")
```

```
usage = float(input("Enter usage minutes: "))
months = int(input("Enter contract duration (months): "))
charge = float(input("Enter monthly charge: "))
calls = int(input("Enter number of support calls: "))

new = pd.DataFrame([[usage, months, charge, calls]], columns=X.columns)
pred, prob = model.predict(new)[0], model.predict_proba(new)[0][1]
print(f"\nChurn probability: {prob:.2f}")
print("Prediction:", "Customer WILL churn" if pred == 1 else "Customer will NOT churn")
```

Output (executed)



Program 28: K-Means clustering for customer segmentation

Question: Accept the shopping features of a new customer and assign the customer to one of the existing segments using K-Means.

Program (source code)

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# ---- customer data: three behaviour groups ----
rng = np.random.default_rng(4)
data = np.vstack([rng.normal([1500, 2, 30], [300, 1, 8], (60, 3)),
                  rng.normal([5000, 6, 80], [600, 1.5, 12], (60, 3)),
                  rng.normal([9000, 12, 150], [900, 2, 20], (60, 3))])
cols = ["annual_spend", "visits_per_month", "avg_order_value"]
customers = pd.DataFrame(data, columns=cols)

scaler = StandardScaler().fit(customers)
kmeans = KMeans(n_clusters=3, n_init=10, random_state=0).fit(scaler.transform(customers))

spend  = float(input("Enter annual spend: "))
visits = float(input("Enter visits per month: "))
order  = float(input("Enter average order value: "))

new = pd.DataFrame([[spend, visits, order]], columns=cols)
segment = kmeans.predict(scaler.transform(new))[0]

print(f"\nNew customer belongs to Segment {segment}")
print("\nSegment profiles (cluster centres):")
centres = pd.DataFrame(scaler.inverse_transform(kmeans.cluster_centers_), columns=cols).round(1)
print(centres)
```

Output (executed)

```
Program 28 - Console Output

Enter annual spend: 5200
Enter visits per month: 6
Enter average order value: 90

New customer belongs to Segment 0

Segment profiles (cluster centres):
   annual_spend  visits_per_month  avg_order_value
0        5219.7               5.8             82.8
1        8917.0              11.9            146.4
2        1533.7               2.1             29.9
```

Program 29: Evaluation metrics: accuracy, precision, recall, F1-score

Question: Load a dataset, let the user choose the feature names and target variable, train a model and display accuracy, precision, recall and F1-score on the test data.

Program (source code)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

df = load_breast_cancer(as_frame=True).frame
print("Available columns (first 6):", list(df.columns[:6]), "... target")

features = [f.strip() for f in input("Enter feature names (comma separated): ").split(",")]
target = input("Enter target variable name: ").strip()

X_train, X_test, y_train, y_test = train_test_split(df[features], df[target],
                                                    test_size=0.25, random_state=1)
model = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nEvaluation on test data")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred):.4f}")
print(f"F1-score : {f1_score(y_test, y_pred):.4f}")
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
```

Output (executed)

```
Program 29 - Console Output

Available columns (first 6): ['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean
smoothness', 'mean compactness'] ... target
Enter feature names (comma separated): mean radius, mean texture, mean area, mean smoothness
Enter target variable name: target

Evaluation on test data
Accuracy : 0.9231
Precision: 0.9140
Recall   : 0.9659
F1-score : 0.9392
Confusion matrix:
 [[47  8]
 [ 3 85]]
```

Program 30: CART (decision tree regressor) for used-car price with decision path

Question: Predict the price of a used car from mileage, age, brand and engine type using CART and display the decision path followed for the new car.

Program (source code)

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeRegressor

# ---- used-car dataset ----
rng = np.random.default_rng(6)
n = 200
cars = pd.DataFrame({"mileage": rng.integers(5000, 150000, n),
                     "age": rng.integers(1, 12, n),
                     "brand": rng.choice(["Toyota", "Honda", "BMW", "Ford"], n),
                     "engine": rng.choice(["Petrol", "Diesel", "Hybrid"], n)})
base = {"Toyota": 9000, "Honda": 8500, "BMW": 16000, "Ford": 7500}
extra = {"Petrol": 0, "Diesel": 800, "Hybrid": 2500}
cars["price"] = (cars["brand"].map(base) + cars["engine"].map(extra)
                 - 0.05 * cars["mileage"] - 400 * cars["age"] + rng.normal(0, 300, n))

X = pd.get_dummies(cars.drop(columns="price"), columns=["brand", "engine"])
tree = DecisionTreeRegressor(max_depth=4, random_state=0).fit(X, cars["price"])

mileage = float(input("Enter mileage (km): "))
age = float(input("Enter age (years): "))
brand = input("Enter brand (Toyota/Honda/BMW/Ford): ")
engine = input("Enter engine type (Petrol/Diesel/Hybrid): ")

new = pd.DataFrame([{"mileage": mileage, "age": age, "brand": brand, "engine": engine}])
new = pd.get_dummies(new, columns=["brand", "engine"]).reindex(columns=X.columns, fill_value=0)
new = new.astype(float)
print(f"\nPredicted price: ${tree.predict(new)[0]:,.2f}")

print("\nDecision path:")
t = tree.tree_
for node in tree.decision_path(new).indices:
    if t.children_left[node] == -1:
        print(f"  leaf node {node}: predicted price = {t.value[node][0][0]:,.2f}")
    else:
        name, thr = X.columns[t.feature[node]], t.threshold[node]
        val = new.iloc[0, t.feature[node]]
        print(f"  node {node}: {name} = {val:g} {'<=' if val <= thr else '> '} {thr:.2f}")
```

Output (executed)

```
Program 30 - Console Output

Enter mileage (km): 45000
Enter age (years): 4
Enter brand (Toyota/Honda/BMW/Ford): Honda
Enter engine type (Petrol/Diesel/Hybrid): Petrol

Predicted price: $5,308.33

Decision path:
  node 0: brand_BMW = 0 <= 0.50
  node 1: mileage = 45000 <= 76165.50
  node 2: age = 4 <= 7.50
  node 3: mileage = 45000 >  32250.50
  leaf node 5: predicted price = 5,308.33
```

Program 31: Customer segmentation using K-Means clustering

Question: Segment e-commerce customers into groups of similar behaviour so that the marketing team can tailor campaigns. Choose the number of clusters, profile each segment and visualize.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(12)
df = pd.DataFrame(np.vstack([rng.normal([25, 30, 80, 25], [4, 6, 8, 5], (70, 4)),
                             rng.normal([40, 85, 60, 14], [6, 10, 10, 4], (70, 4)),
                             rng.normal([55, 50, 25, 5], [6, 8, 8, 2], (70, 4))]),
                  columns=["age", "annual_income_k", "spending_score", "purchases_per_year"])

X = StandardScaler().fit_transform(df)
print("Silhouette score for different k:")
scores = {}
for k in range(2, 7):
    scores[k] = silhouette_score(X, KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X))
    print(f"  k = {k}: {scores[k]:.3f}")
best_k = max(scores, key=scores.get)
print("Best number of clusters:", best_k)

df["segment"] = KMeans(n_clusters=best_k, n_init=10, random_state=0).fit_predict(X)
print("\nSegment profile (mean values):")
print(df.groupby("segment").mean().round(1).assign(customers=df["segment"].value_counts().sort_index()))

plt.figure(figsize=(6.5, 4))
plt.scatter(df["annual_income_k"], df["spending_score"], c=df["segment"], cmap="viridis")
plt.title("Customer Segments"); plt.xlabel("Annual income (k)"); plt.ylabel("Spending score")
plt.show()
```

Output (executed)

Program 31 - Console Output

```
Silhouette score for different k:
  k = 2: 0.551
  k = 3: 0.640
  k = 4: 0.514
  k = 5: 0.407
  k = 6: 0.271
Best number of clusters: 3

Segment profile (mean values):
           age  annual_income_k  spending_score  purchases_per_year  customers
segment
0         40.6             84.8            59.5                14.9         70
1         25.2             29.2            80.1                25.8         70
2         54.6             48.9            24.3                 5.1         70
```



Program 32: Bivariate analysis and simple linear regression (house price vs size)

Question: Perform bivariate analysis and build a linear regression model to predict house price from house size; evaluate the model.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

rng = np.random.default_rng(3)
size = rng.integers(500, 4000, 120)
df = pd.DataFrame({"size": size, "price": 45 * size + 20000 + rng.normal(0, 25000, 120)})

print("Correlation between size and price: %.3f" % df["size"].corr(df["price"]))

X_train, X_test, y_train, y_test = train_test_split(df[["size"]], df["price"],
                                                    test_size=0.2, random_state=0)
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)

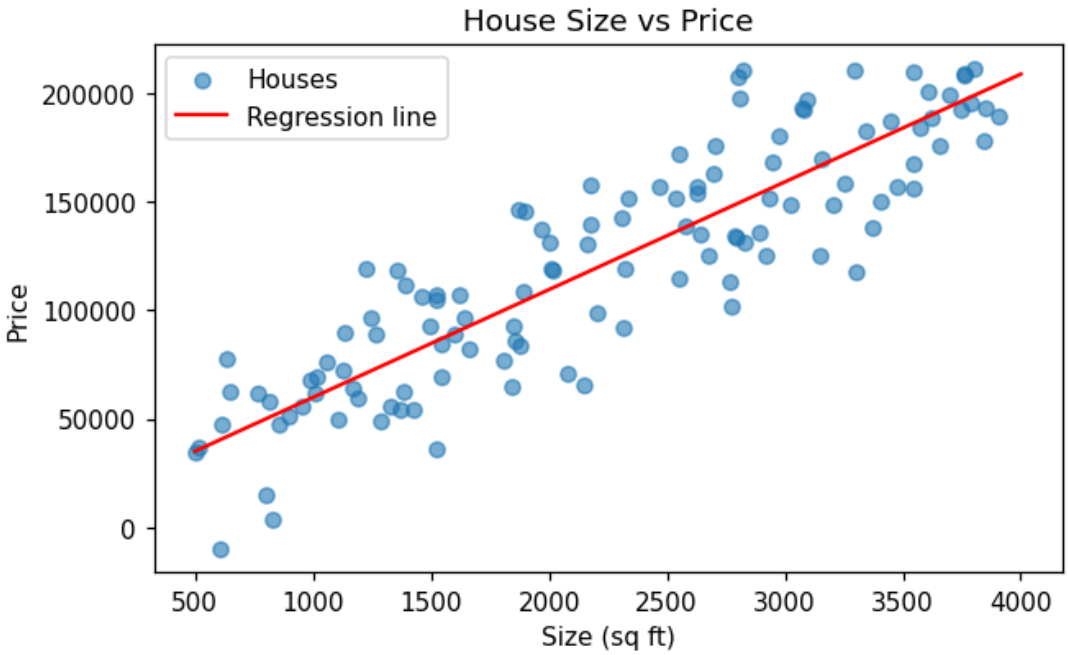
print(f"Equation : price = {model.coef_[0]:.2f} * size + {model.intercept_:.2f}")
print(f"R-squared: {r2_score(y_test, pred):.4f}")
print(f"MAE      : {mean_absolute_error(y_test, pred):,.2f}")
print(f"RMSE     : {np.sqrt(mean_squared_error(y_test, pred)):,.2f}")

plt.figure(figsize=(6.5, 3.8))
plt.scatter(df["size"], df["price"], alpha=0.6, label="Houses")
xs = np.linspace(500, 4000, 50)
plt.plot(xs, model.coef_[0] * xs + model.intercept_, color="red", label="Regression line")
plt.title("House Size vs Price"); plt.xlabel("Size (sq ft)"); plt.ylabel("Price"); plt.legend()
plt.show()
```

Output (executed)

Program 32 - Console Output

```
Correlation between size and price: 0.882
Equation : price = 49.70 * size + 10231.21
R-squared: 0.7126
MAE      : 22,005.83
RMSE     : 25,502.37
```



Program 33: Multiple linear regression for car price with influential factors

Question: Predict car prices from engine size, horsepower and fuel efficiency, evaluate the model and report the most influential factors for the marketing team.

Program (source code)

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error

rng = np.random.default_rng(9)
n = 200
cars = pd.DataFrame({"engine_size": rng.uniform(1.0, 5.0, n),
                     "horsepower": rng.uniform(70, 400, n),
                     "mpg": rng.uniform(15, 45, n)})
cars["price"] = (3000 * cars["engine_size"] + 90 * cars["horsepower"]
                 - 150 * cars["mpg"] + 8000 + rng.normal(0, 2500, n))

X, y = cars.drop(columns="price"), cars["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

scaler = StandardScaler().fit(X_train)
model = LinearRegression().fit(scaler.transform(X_train), y_train)
pred = model.predict(scaler.transform(X_test))

print(f"R-squared: {r2_score(y_test, pred):.4f}")
print(f"RMSE     : {np.sqrt(mean_squared_error(y_test, pred)):,.2f}")

importance = pd.Series(model.coef_, index=X.columns).sort_values(key=abs, ascending=False)
print("\nStandardised coefficients (influence on price):")
print(importance.round(2))
print(f"\nInsight: '{importance.index[0]}' is the most influential factor affecting car price.")
```

Output (executed)

```
Program 33 - Console Output

R-squared: 0.9453
RMSE     : 2,260.46

Standardised coefficients (influence on price):
horsepower     8429.34
engine_size    3584.78
mpg           -1384.98
dtype: float64

Insight: 'horsepower' is the most influential factor affecting car price.
```

Program 34: KNN classifier for treatment outcome (Good / Bad)

Question: Build a KNN classifier to predict the treatment outcome of patients, evaluate with accuracy, precision, recall and F1-score, and display test-set predictions.

Program (source code)

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

rng = np.random.default_rng(14)
n = 200
df = pd.DataFrame({"age": rng.integers(25, 80, n), "gender": rng.integers(0, 2, n),
                   "blood_pressure": rng.normal(130, 18, n), "cholesterol": rng.normal(210, 35, n)})
risk = 0.03 * (df["age"] - 50) + 0.03 * (df["blood_pressure"] - 130) + 0.01 * (df["cholesterol"] - 210)
df["outcome"] = np.where(risk + rng.normal(0, 0.6, n) > 0, "Bad", "Good")

X, y = df.drop(columns="outcome"), df["outcome"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=1, stratify=y)
scaler = StandardScaler().fit(X_train)
knn = KNeighborsClassifier(n_neighbors=5).fit(scaler.transform(X_train), y_train)
pred = knn.predict(scaler.transform(X_test))

print(f"Accuracy : {accuracy_score(y_test, pred):.3f}")
print(f"Precision: {precision_score(y_test, pred, pos_label='Good'):.3f}")
print(f"Recall   : {recall_score(y_test, pred, pos_label='Good'):.3f}")
print(f"F1-score : {f1_score(y_test, pred, pos_label='Good'):.3f}")

results = X_test.copy().round(1)
results["actual"], results["predicted"] = y_test, pred
print("\nFirst 8 test-set predictions:")
print(results.head(8).to_string())
```

Output (executed)

```
Program 34 - Console Output

Accuracy : 0.720
Precision: 0.765
Recall   : 0.565
F1-score : 0.650

First 8 test-set predictions:
     age  gender  blood_pressure  cholesterol actual predicted
199   38       1           131.5        206.7   Good      Good
35    60       1           129.6        254.2    Bad       Bad
158   36       1           152.0        157.6   Good       Bad
161   28       0           138.8        215.0   Good      Good
24    25       0           135.9        165.3   Good      Good
187   44       0           123.4        160.7   Good      Good
47    48       1           113.0        200.5   Good      Good
16    66       0           139.0        232.6    Bad       Bad
```

Program 35: K-Means clustering of retail customers by spending pattern

Question: Group retail customers into segments using total amount spent and visit frequency with K-Means.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(15)
data = np.vstack([rng.normal([800, 3], [200, 1], (50, 2)),
                  rng.normal([3500, 9], [500, 2], (50, 2)),
                  rng.normal([7500, 20], [900, 3], (50, 2))])
df = pd.DataFrame(data, columns=["total_spent", "visit_frequency"])
df.insert(0, "customer_id", range(1, len(df) + 1))

X = StandardScaler().fit_transform(df[["total_spent", "visit_frequency"]])

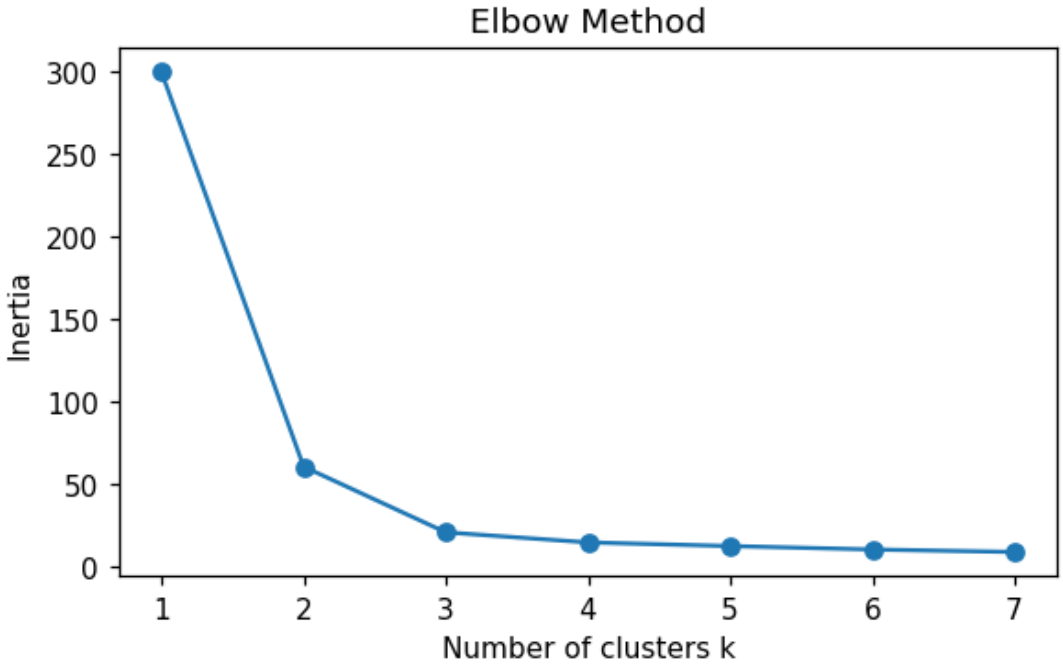
inertia = [KMeans(n_clusters=k, n_init=10, random_state=0).fit(X).inertia_ for k in range(1, 8)]
plt.figure(figsize=(6, 3.4))
plt.plot(range(1, 8), inertia, marker="o")
plt.title("Elbow Method"); plt.xlabel("Number of clusters k"); plt.ylabel("Inertia")
plt.show()

df["segment"] = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
summary = df.groupby("segment").agg(customers=("customer_id", "count"),
                                    avg_spent=("total_spent", "mean"),
                                    avg_visits=("visit_frequency", "mean")).round(1)
print("Customer segments (k = 3):")
print(summary)
```

Output (executed)

```
Program 35 - Console Output

Customer segments (k = 3):
         customers  avg_spent  avg_visits
segment
0               50     3490.4         9.1
1               50     7434.5        20.1
2               50      834.0         3.0
```



Program 36: Variability of stock prices from a CSV file

Question: Read stock closing prices from a CSV file, calculate the variability of prices and provide insights.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ---- create stock_prices.csv (60 trading days) ----
rng = np.random.default_rng(20)
close = 150 * np.cumprod(1 + rng.normal(0.001, 0.015, 60))
pd.DataFrame({"Date": pd.bdate_range("2026-01-01", periods=60), "Close": close.round(2)}
             ).to_csv("stock_prices.csv", index=False)

df = pd.read_csv("stock_prices.csv", parse_dates=["Date"])
df["Return"] = df["Close"].pct_change() * 100

print(f"Mean closing price : {df['Close'].mean():.2f}")
print(f"Variance           : {df['Close'].var():.2f}")
print(f"Standard deviation : {df['Close'].std():.2f}")
print(f"Price range        : {df['Close'].min():.2f} - {df['Close'].max():.2f}")
print(f"Coefficient of var.: {df['Close'].std() / df['Close'].mean() * 100:.2f}%")
print(f"Daily return std   : {df['Return'].std():.2f}%")
best, worst = df.loc[df['Return'].idxmax()], df.loc[df['Return'].idxmin()]
print(f"Best day  : {best['Date'].date()} ({best['Return']:+.2f}%)")
print(f"Worst day : {worst['Date'].date()} ({worst['Return']:+.2f}%)")

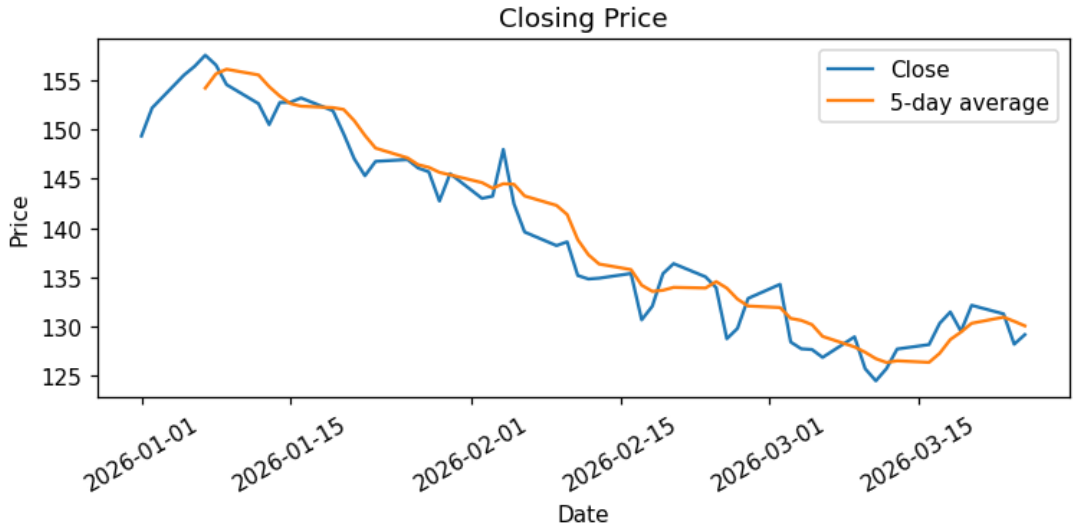
plt.figure(figsize=(7, 3.6))
plt.plot(df["Date"], df["Close"], label="Close")
plt.plot(df["Date"], df["Close"].rolling(5).mean(), label="5-day average")
```

```
plt.title("Closing Price"); plt.xlabel("Date"); plt.ylabel("Price"); plt.legend(); plt.xticks(rotation=30)
plt.tight_layout(); plt.show()
```

Output (executed)

```
Program 36 - Console Output

Mean closing price : 139.30
Variance           : 99.71
Standard deviation : 9.99
Price range        : 124.49 - 157.55
Coefficient of var.: 7.17%
Daily return std   : 1.68%
Best day  : 2026-02-04 (+3.31%)
Worst day : 2026-03-03 (-4.36%)
```



Program 37: Correlation between study time and exam scores

Question: Identify the correlation between study time and exam score and explore several plotting functions.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(30)
hours = rng.uniform(1, 10, 60)
df = pd.DataFrame({"study_hours": hours,
                   "exam_score": np.clip(35 + 6 * hours + rng.normal(0, 6, 60), 0, 100)})

pearson = df["study_hours"].corr(df["exam_score"])
spearman = df["study_hours"].corr(df["exam_score"], method="spearman")
print(f"Pearson correlation : {pearson:.3f}")
print(f"Spearman correlation: {spearman:.3f}")
print("Interpretation:", "strong positive correlation" if pearson > 0.7 else "weak/moderate correlation")

fig, ax = plt.subplots(2, 2, figsize=(8, 6))
ax[0, 0].scatter(df["study_hours"], df["exam_score"]); ax[0, 0].set_title("Scatter plot")

m, c = np.polyfit(df["study_hours"], df["exam_score"], 1)
xs = np.linspace(1, 10, 50)
ax[0, 1].scatter(df["study_hours"], df["exam_score"], alpha=0.5)
ax[0, 1].plot(xs, m * xs + c, color="red"); ax[0, 1].set_title("Scatter + regression line")

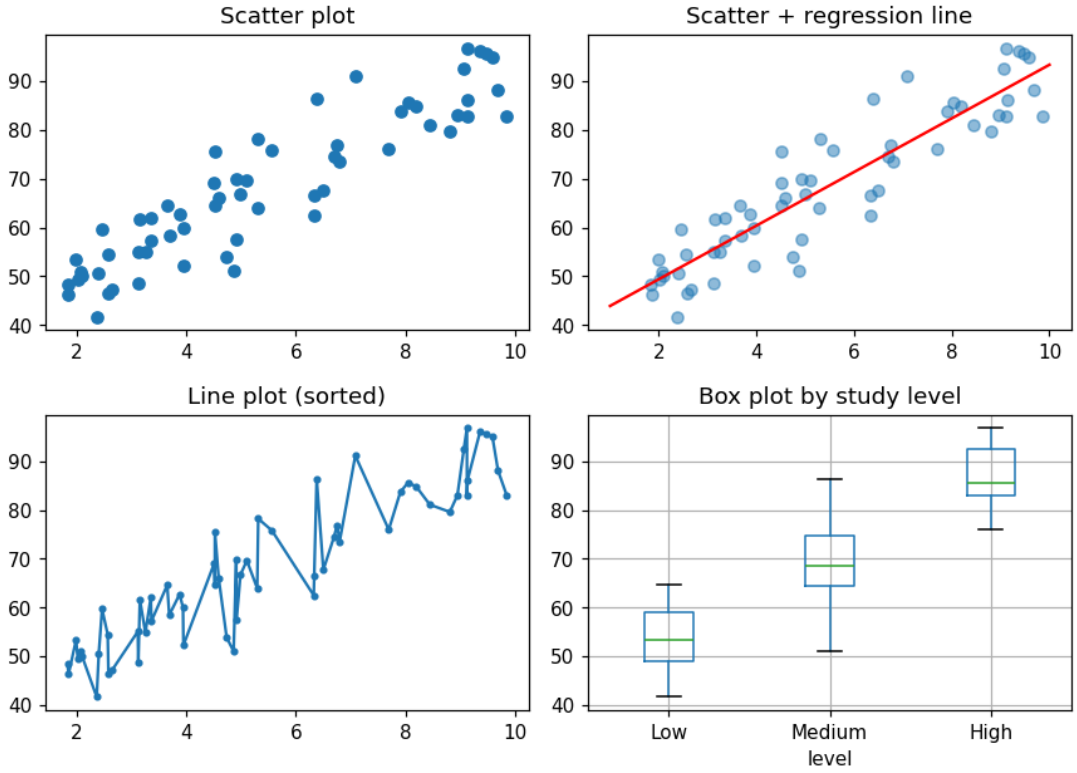
s = df.sort_values("study_hours")
ax[1, 0].plot(s["study_hours"], s["exam_score"], marker=".")
ax[1, 0].set_title("Line plot (sorted)")
```

```
df["level"] = pd.cut(df["study_hours"], [0, 4, 7, 10], labels=["Low", "Medium", "High"])
df.boxplot(column="exam_score", by="level", ax=ax[1, 1]); ax[1, 1].set_title("Box plot by study level")
plt.suptitle(""); plt.tight_layout(); plt.show()
```

Output (executed)

```
Program 37 - Console Output

Pearson correlation : 0.915
Spearman correlation: 0.914
Interpretation: strong positive correlation
```



Program 38: Temperature variability analysis for different cities

Question: For daily temperatures of each city over a year: mean, standard deviation, city with the highest range and the city with the most consistent temperature.

Program (source code)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(40)
days = np.arange(365)
season = np.sin(2 * np.pi * (days - 80) / 365)
temps = pd.DataFrame({
    "Chennai":   31 + 3 * season + rng.normal(0, 1.5, 365),
    "Delhi":     25 + 12 * season + rng.normal(0, 2.5, 365),
    "Mumbai":    28 + 2.5 * season + rng.normal(0, 1.2, 365),
    "Bengaluru": 25 + 4 * season + rng.normal(0, 1.8, 365)})

mean_t = temps.mean()
std_t = temps.std()
range_t = temps.max() - temps.min()

print("1. Mean temperature (C):\n", mean_t.round(2))
print("\n2. Standard deviation:\n", std_t.round(2))
print(f"\n3. Highest temperature range: {range_t.idxmax()} ({range_t.max():.2f} C)")
print(f"4. Most consistent temperature: {std_t.idxmin()} (std = {std_t.min():.2f})")
```

Output (executed)

```
Program 38 - Console Output

1. Mean temperature (C):
 Chennai      30.98
Delhi        24.96
Mumbai       27.94
Bengaluru    25.03
dtype: float64

2. Standard deviation:
 Chennai      2.78
Delhi        8.80
Mumbai       2.17
Bengaluru    3.49
dtype: float64

3. Highest temperature range: Delhi (35.97 C)
4. Most consistent temperature: Mumbai (std = 2.17)
```

Program 39: K-Means customer clustering with scatter-plot visualization

Question: Group customers by amount spent and items purchased using K-Means and visualize the clusters and centroids with a scatter plot.

Program (source code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

rng = np.random.default_rng(50)
data = np.vstack([rng.normal([40, 2], [12, 1], (60, 2)),
                  rng.normal([150, 8], [25, 2], (60, 2)),
                  rng.normal([320, 18], [40, 3], (60, 2))])
df = pd.DataFrame(data, columns=["amount_spent", "items_purchased"])

km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(df)
df["cluster"] = km.labels_
print("Customers per cluster:")
print(df["cluster"].value_counts().sort_index())
print("\nCluster centres (amount_spent, items_purchased):")
print(np.round(km.cluster_centers_, 1))

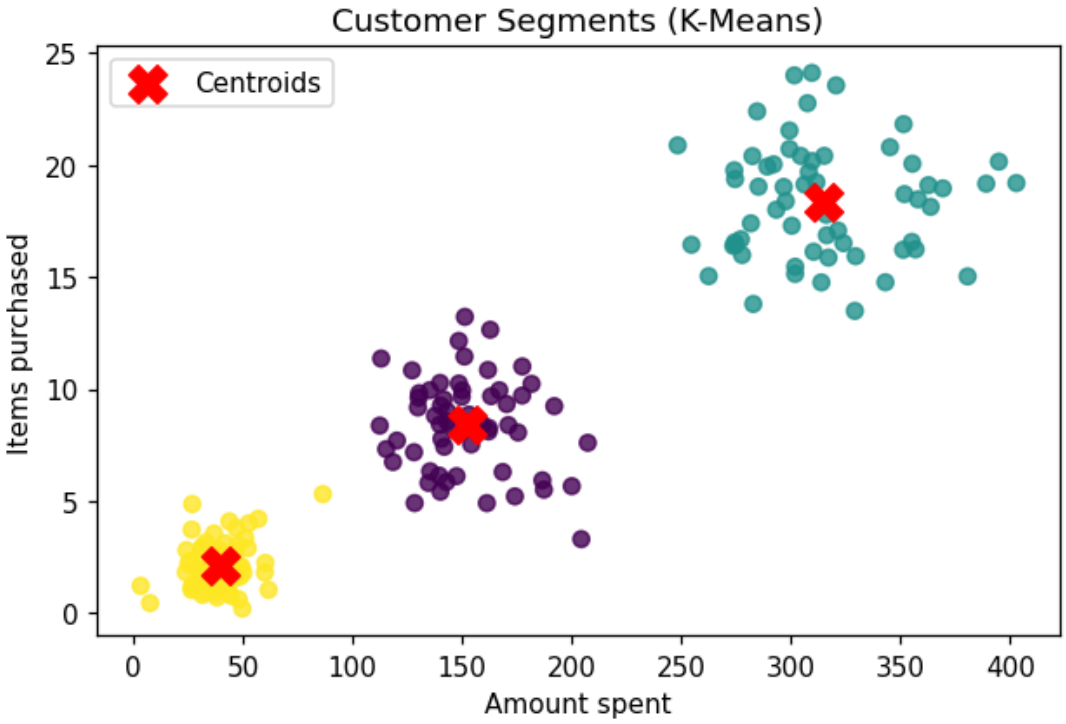
plt.figure(figsize=(6.5, 4))
plt.scatter(df["amount_spent"], df["items_purchased"], c=df["cluster"], cmap="viridis", alpha=0.8)
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1],
            c="red", marker="X", s=200, label="Centroids")
plt.title("Customer Segments (K-Means)"); plt.xlabel("Amount spent"); plt.ylabel("Items purchased")
plt.legend(); plt.show()
```

Output (executed)

```
Program 39 - Console Output

Customers per cluster:
cluster
0    59
1    60
2    61
Name: count, dtype: int64

Cluster centres (amount_spent, items_purchased):
[[152.2   8.4]
 [314.7  18.4]
 [ 39.4   2.2]]
```



Program 40: Soccer players analysis (CSV, Pandas, bar chart)

Question: Create a players CSV; find the top 5 goal scorers and top 5 salaries, the average age, players above the average age, and plot the distribution of players by position.

Program (source code)

```
import pandas as pd
import matplotlib.pyplot as plt

players = pd.DataFrame({
    "name": ["Arjun", "Luis", "Marco", "Sam", "Kenji", "Omar", "Diego", "Ravi",
             "Tom", "Ivan", "Leo", "Hugo", "Nico", "Paulo", "Yusuf"],
    "age": [24, 31, 28, 22, 35, 27, 30, 26, 33, 29, 21, 34, 25, 32, 23],
    "position": ["Forward", "Midfielder", "Defender", "Forward", "Goalkeeper", "Midfielder", "Forward",
                 "Defender", "Defender", "Midfielder", "Forward", "Goalkeeper", "Midfielder",
                 "Forward", "Defender"],
    "goals": [18, 9, 3, 14, 0, 7, 21, 2, 4, 11, 12, 0, 6, 16, 1],
    "weekly_salary": [45000, 62000, 30000, 38000, 28000, 41000, 75000, 26000, 31000,
                      52000, 33000, 29000, 36000, 68000, 24000]})
players.to_csv("players.csv", index=False)

df = pd.read_csv("players.csv")
top_goals = df.nlargest(5, "goals")[["name", "goals"]]
top_pay = df.nlargest(5, "weekly_salary")[["name", "weekly_salary"]]
print("Top 5 goal scorers:\n", top_goals.to_string(index=False))
print("\nTop 5 highest salaries:\n", top_pay.to_string(index=False))

avg_age = df["age"].mean()
print(f"\nAverage age of players: {avg_age:.2f}")
print("Players above average age:", ", ".join(df.loc[df["age"] > avg_age, "name"]))

df["position"].value_counts().plot(kind="bar", color="seagreen", figsize=(6, 3.4))
plt.title("Players by Position"); plt.xlabel("Position"); plt.ylabel("Number of players")
plt.xticks(rotation=0); plt.tight_layout(); plt.show()
```

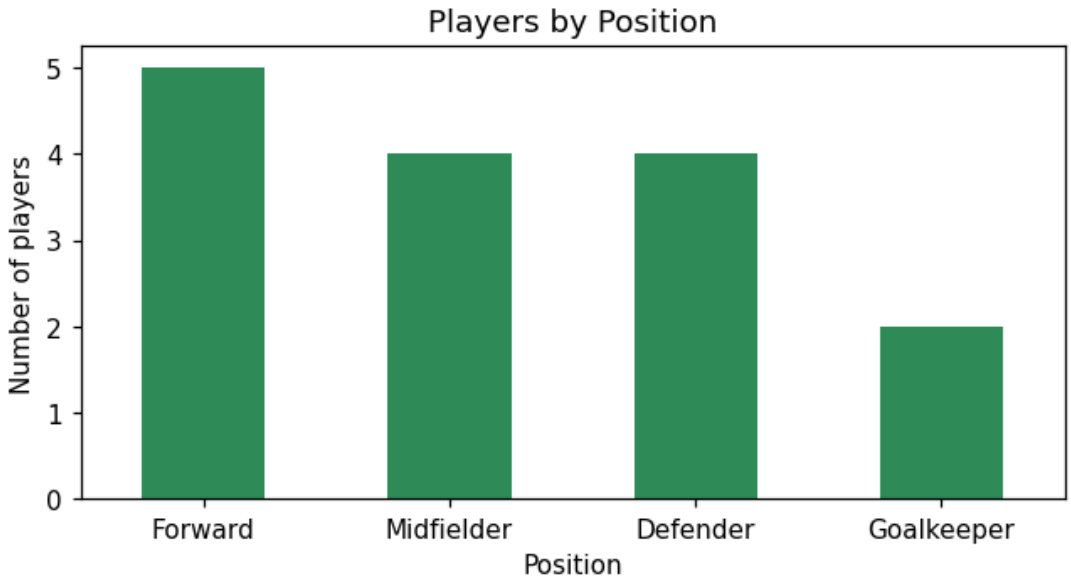
Output (executed)

Program 40 - Console Output

```
Top 5 goal scorers:
  name  goals
Diego     21
Arjun     18
Paulo     16
  Sam     14
  Leo     12

Top 5 highest salaries:
  name  weekly_salary
Diego          75000
Paulo          68000
 Luis          62000
 Ivan          52000
Arjun          45000

Average age of players: 28.00
Players above average age: Luis, Kenji, Diego, Tom, Ivan, Hugo, Paulo
```


Players by Position
Number of players
0
1
2
3
4
5
Forward
Midfielder
Defender
Goalkeeper
Position